



3.3 Rest Parameters and Function Overloads

Mark as complete

Rest Parameters and Function Overloads

In this lesson, you'll learn two powerful TypeScript features for creating flexible functions: rest parameters for handling variable numbers of arguments, and function overloads for providing multiple call signatures based on different input types.



Learning Objectives

By the end of this lesson, you will be able to:

- Use rest parameters to accept variable numbers of arguments
- Type rest parameters correctly with arrays and tuples
- Create function overloads for different call signatures
- Understand when to use overloads vs union types
- Combine rest parameters with other parameter types



Why These Features Matter

Real-World Analogy:

Rest Parameters are like a shopping basket - you can put in as many items as you want, and the basket handles them all as a collection. When you calculate the total, you process all items regardless of how many there are.

Function Overloads are like a multi-tool that adapts to what you're trying to do. Use it with a screw, and it's a screwdriver. Use it with a nail, and it's a hammer. The tool knows how to behave based on what you give it.



1 Rest Parameters Basics

Rest parameters let you accept any number of arguments as an array.

```

2 function sum(...numbers: number[]): number {
3     return numbers.reduce((total, num) => total + num, 0);
4 }
5
6 console.log(sum(1));           // 1
7 console.log(sum(1, 2));        // 3
8 console.log(sum(1, 2, 3, 4));  // 10
9 console.log(sum());            // 0 (empty array)

```

Rest Parameters with Other Parameters

Rest parameters must come last:

```

1 // ✅ Correct: Rest parameter at end
2 function greetAll(greeting: string, ...names: string[]): string {
3     return names.map(name => `${greeting}, ${name}!`).join(" ");
4 }
5
6 console.log(greetAll("Hello", "Alice", "Bob", "Charlie"));
7 // "Hello, Alice! Hello, Bob! Hello, Charlie!"
8
9 // ❌ Error: Rest parameter must be last
10 function invalid(...numbers: number[], suffix: string): string {
11     //             ^^^^^^
12     // Error: A rest parameter must be last in a parameter list
13     return "";
14 }

```

Mixed Parameter Types

```

1 function createMessage(
2     prefix: string,
3     suffix: string = "!",
4     ...words: string[]
5 ): string {
6     return `${prefix} ${words.join(" ")}${suffix}`;
7 }

```

```
11
12 console.log(createMessage("Message", undefined, "test", "one", "two"));
13 // "Message test one two!"
```

2 Typing Rest Parameters

Array Rest Parameters

```
1 // Basic array type
2 function multiply(...numbers: number[]): number {
3     return numbers.reduce((product, num) => product * num, 1);
4 }
5
6 // Union type array
7 function process(...items: (string | number][]): void {
8     items.forEach(item => {
9         if (typeof item === "string") {
10             console.log(`String: ${item.toUpperCase()}`);
11         } else {
12             console.log(`Number: ${item * 2}`);
13         }
14     });
15 }
16
17 process("hello", 42, "world", 100);
18 // String: HELLO
19 // Number: 84
20 // String: WORLD
21 // Number: 200
```

Tuple Rest Parameters

Use tuple types for more specific rest parameter patterns:

```
1 // Exactly 3 numbers
2 function addThree(...numbers: [number, number, number]): number {
```

```
6  addThree(1, 2, 3);      // ✅ OK
7  addThree(1, 2);        // ❌ Error: Expected 3 arguments
8  addThree(1, 2, 3, 4);  // ❌ Error: Expected 3 arguments
9
10 // At least 2 numbers, then any number of additional
11 function sumMinTwo(...numbers: [number, number, ...number[]]): number {
12     return numbers.reduce((sum, num) => sum + num, 0);
13 }
14
15 sumMinTwo(1, 2);        // ✅ OK
16 sumMinTwo(1, 2, 3, 4); // ✅ OK
17 sumMinTwo(1);           // ❌ Error: Need at least 2 arguments
```

Readonly Rest Parameters

```
1  function display(...items: readonly string[]): void {
2      // Cannot modify items array
3      // items.push("new"); // ❌ Error: Property 'push' does not exist
4
5      console.log(items.join(", "));
6  }
7
8  display("apple", "banana", "cherry");
```

3 The Spread Operator with Rest Parameters

Rest parameters work seamlessly with the spread operator:

Spreading Arrays into Functions

```
1  function calculateTotal(...prices: number[]): number {
2      return prices.reduce((sum, price) => sum + price, 0);
3  }
4
5  const priceList1 = [10, 20, 30];
6  const priceList2 = [5, 15];
```

```
10 console.log(calculateTotal(100, ...priceList2, 200)); // 320
```

Building Flexible APIs

```
1 function merge<T>(...arrays: T[][]): T[] {
2   return arrays.flat();
3 }
4
5 const numbers1 = [1, 2, 3];
6 const numbers2 = [4, 5];
7 const numbers3 = [6, 7, 8, 9];
8
9 console.log(merge(numbers1, numbers2, numbers3));
10 // [1, 2, 3, 4, 5, 6, 7, 8, 9]
11
12 const names1 = ["Alice", "Bob"];
13 const names2 = ["Charlie"];
14
15 console.log(merge(names1, names2));
16 // ["Alice", "Bob", "Charlie"]
```

4 Function Overloads

Function overloads let you define multiple call signatures for the same function, providing better type checking and autocomplete.

Basic Overload Syntax

```
1 // Overload signatures
2 function format(value: string): string;
3 function format(value: number): string;
4 function format(value: boolean): string;
5
6 // Implementation signature (must be compatible with all overloads)
7 function format(value: string | number | boolean): string {
8   if (typeof value === "string") {
```

```
12     } else {
13         return value ? "Yes" : "No";
14     }
15 }
16
17 console.log(format("hello")); // "hello"
18 console.log(format(42));      // "42.00"
19 console.log(format(true));    // "Yes"
20
21 // TypeScript knows the return type for each call
22 const str: string = format("test"); // Type is string
23 const num: string = format(123);    // Type is string
24 const bool: string = format(false); // Type is string
```

Overloads with Different Return Types

```
1 // Different return types based on input
2 function getValue(key: "name"): string;
3 function getValue(key: "age"): number;
4 function getValue(key: "active"): boolean;
5 function getValue(key: string): string | number | boolean {
6     const data: Record<string, string | number | boolean> = {
7         name: "Alice",
8         age: 30,
9         active: true
10    };
11    return data[key];
12 }
13
14 // TypeScript infers correct return type
15 const name: string = getValue("name"); // ✅ string
16 const age: number = getValue("age");   // ✅ number
17 const active: boolean = getValue("active"); // ✅ boolean
18
19 // const wrong: number = getValue("name"); // ❌ Error: Type 'string' is not
```

```
2 function makeArray(length: number): number[];
3 // Overload with two parameters
4 function makeArray(length: number, value: number): number[];
5
6 // Implementation
7 function makeArray(length: number, value?: number): number[] {
8   if (value !== undefined) {
9     return Array(length).fill(value);
10  }
11  return Array(length).fill(0);
12 }
13
14 const arr1 = makeArray(5);      // [0, 0, 0, 0, 0]
15 const arr2 = makeArray(3, 7);   // [7, 7, 7]
```

5 Practical Overload Examples

Example 1: Flexible Search Function

```
1 interface User {
2   id: number;
3   name: string;
4   email: string;
5 }
6
7 const users: User[] = [
8   { id: 1, name: "Alice", email: "alice@example.com" },
9   { id: 2, name: "Bob", email: "bob@example.com" },
10  { id: 3, name: "Charlie", email: "charlie@example.com" }
11 ];
12
13 // Find by ID
14 function findUser(id: number): User | undefined;
15 // Find by email
16 function findUser(email: string): User | undefined;
17 // Find by predicate
```

```
21 function findUser(  
22   criteria: number | string | ((user: User) => boolean)  
23 ): User | undefined {  
24   if (typeof criteria === "number") {  
25     return users.find(user => user.id === criteria);  
26   } else if (typeof criteria === "string") {  
27     return users.find(user => user.email === criteria);  
28   } else {  
29     return users.find(criteria);  
30   }  
31 }  
32  
33 // Usage with type safety  
34 const user1 = findUser(1); // Find by ID  
35 const user2 = findUser("bob@example.com"); // Find by email  
36 const user3 = findUser(u => u.name === "Charlie"); // Find by predicate
```

Example 2: Event Listener with Overloads

```
1 type EventHandler = (event: Event) => void;  
2  
3 interface EventMap {  
4   click: MouseEvent;  
5   keypress: KeyboardEvent;  
6   scroll: UIEvent;  
7   custom: CustomEvent;  
8 }  
9  
10 // Overloads for specific event types  
11 function addEventListener(  
12   type: "click",  
13   handler: (event: MouseEvent) => void  
14 ): void;  
15 function addEventListener(  
16   type: "keypress",  
17   handler: (event: KeyboardEvent) => void  
18 ): void;
```

```
22 ): void;  
23 function addEventListener(  
24   type: string,  
25   handler: (event: Event) => void  
26 ): void;  
27  
28 // Implementation  
29 function addEventListener(  
30   type: string,  
31   handler: (event: Event) => void  
32 ): void {  
33   console.log(`Adding ${type} event listener`);  
34   // Actual implementation would register the event  
35 }  
36  
37 // Type-safe usage  
38 addEventListener("click", (e) => {  
39   // e is typed as MouseEvent  
40   console.log(e.clientX, e.clientY);  
41 });  
42  
43 addEventListener("keypress", (e) => {  
44   // e is typed as KeyboardEvent  
45   console.log(e.key);  
46 });
```

Example 3: createElement with Overloads

```
1 interface HTMLElementMap {  
2   div: HTMLDivElement;  
3   span: HTMLSpanElement;  
4   input: HTMLInputElement;  
5   button: HTMLButtonElement;  
6 }  
7  
8 // Overload for each element type  
9 function createElement(tag: "div"): HTMLDivElement;
```

```

13 function createElement(tag: string): HTMLElement;
14
15 // Implementation
16 function createElement(tag: string): HTMLElement {
17     return document.createElement(tag);
18 }
19
20 // TypeScript knows the exact return type
21 const div = createElement("div");           // HTMLDivElement
22 const input = createElement("input");       // HTMLInputElement
23
24 // Can use element-specific properties
25 div.style.display = "block";                // OK
26 input.value = "test";                       // OK
27 // div.value = "test";                      // Error: Property 'value' does not

```

6 Overloads vs Union Types

When should you use overloads instead of union types?

Use Union Types When...

Simple relationship between input and output:

```

1 // ✅ Good use of union type
2 function format(value: string | number): string {
3     return typeof value === "string" ? value : value.toString();
4 }

```

Use Overloads When...

Complex relationships between parameters and return types:

```

1 // ✅ Good use of overloads
2 function parse(value: string, type: "number"): number;

```

```
6 function parse(  
7   value: string,  
8   type: "number" | "boolean" | "date"  
9 ): number | boolean | Date {  
10  switch (type) {  
11    case "number":  
12      return parseFloat(value);  
13    case "boolean":  
14      return value === "true";  
15    case "date":  
16      return new Date(value);  
17  }  
18 }  
19  
20 // Return type is precise  
21 const num: number = parse("42", "number");  
22 const bool: boolean = parse("true", "boolean");  
23 const date: Date = parse("2024-01-01", "date");
```

7 Combining Rest Parameters and Overloads

You can use rest parameters in overload signatures:

```
1 // Overload with rest parameters  
2 function combine(separator: string, ...values: string[]): string;  
3 function combine(separator: string, ...values: number[]): string;  
4  
5 // Implementation  
6 function combine(  
7   separator: string,  
8   ...values: (string | number)[]  
9 ): string {  
10  return values.join(separator);  
11 }  
12
```

Complex Example: Query Builder

```
1  type QueryValue = string | number | boolean;
2
3  // Overload: Single key-value
4  function buildQuery(key: string, value: QueryValue): string;
5  // Overload: Object of key-values
6  function buildQuery(params: Record<string, QueryValue>): string;
7  // Overload: Rest parameters
8  function buildQuery(...pairs: [string, QueryValue][]): string;
9
10 // Implementation
11 function buildQuery(
12   keyOrParams: string | Record<string, QueryValue> | [string, QueryValue],
13   value?: QueryValue,
14   ...rest: [string, QueryValue][]
15 ): string {
16   let params: Record<string, QueryValue> = {};
17
18   if (typeof keyOrParams === "string" && value !== undefined) {
19     // Single key-value
20     params[keyOrParams] = value;
21   } else if (typeof keyOrParams === "object" && !Array.isArray(keyOrParams)) {
22     // Object of key-values
23     params = keyOrParams;
24   } else if (Array.isArray(keyOrParams)) {
25     // Rest parameters (array of tuples)
26     [keyOrParams, ...rest].forEach(([key, val]) => {
27       params[key] = val;
28     });
29   }
30
31   return Object.entries(params)
32     .map(([k, v]) => `${k}=${encodeURIComponent(String(v))}`)
33     .join("&");
34 }
35
```

```
39
40 console.log(buildQuery({ name: "Bob", age: 30 }));
41 // "name=Bob&age=30"
42
43 console.log(buildQuery(["name", "Charlie"], ["active", true], ["score", 95]));
44 // "name=Charlie&active=true&score=95"
```

Best Practices

1. Order Overloads from Most Specific to Most General

```
1 // ✅ Good: Most specific first
2 function process(value: "special"): string;
3 function process(value: string): number;
4 function process(value: string): string | number {
5     if (value === "special") {
6         return "Special handling";
7     }
8     return value.length;
9 }
10
11 // ❌ Avoid: General overload first can hide specific ones
12 function badProcess(value: string): number;
13 function badProcess(value: "special"): string; // This might never be called
14 function badProcess(value: string): string | number {
15     // Implementation
16     return 0;
17 }
```

2. Implementation Signature Must Be Compatible

```
1 // ✅ Good: Implementation handles all overloads
2 function good(x: number): number;
3 function good(x: string): string;
```

```
7 }  
8  
9 // ❌ Error: Implementation signature not compatible  
10 function bad(x: number): number;  
11 function bad(x: string): string;  
12 function bad(x: number): number { // ❌ Missing string case  
13     return x * 2;  
14 }
```

3. Use Rest Parameters Judiciously

```
1 // ✅ Good: Clear purpose  
2 function sum(...numbers: number[]): number {  
3     return numbers.reduce((a, b) => a + b, 0);  
4 }  
5  
6 // ❌ Avoid: Mixing too many types  
7 function confusing(  
8     required: string,  
9     optional?: number,  
10    ...rest: (string | number | boolean)[]  
11 ): any {  
12     // Hard to understand and use correctly  
13 }
```



Practice Exercise

Create a `fetch` wrapper function with overloads:

```
1 interface FetchOptions {  
2     method?: "GET" | "POST" | "PUT" | "DELETE";  
3     headers?: Record<string, string>;  
4     body?: any;  
5 }  
6
```

```
10 // Overload 2: URL and options
11 function fetch(url: string, options: FetchOptions): Promise<Response>;
12
13 // Overload 3: URL, method, and body
14 function fetch(
15     url: string,
16     method: "POST" | "PUT",
17     body: any
18 ): Promise<Response>;
19
20 // Your implementation here
21 function fetch(
22     url: string,
23     optionsOrMethod?: FetchOptions | "POST" | "PUT",
24     body?: any
25 ): Promise<Response> {
26     // Implement the function
27 }
```

Requirements:

Implement all three overload patterns

Handle GET, POST, PUT, DELETE methods

Set appropriate default headers

Return a Promise (you can simulate with resolved values)

<details> <summary>Solution</summary>

```
1 interface FetchOptions {
2     method?: "GET" | "POST" | "PUT" | "DELETE";
3     headers?: Record<string, string>;
4     body?: any;
5 }
6
7 interface Response {
8     status: number;
9     data: any;
10 }
```

```
14 function fetch(url: string, options: FetchOptions): Promise<Response>;
15 function fetch(
16     url: string,
17     method: "POST" | "PUT",
18     body: any
19 ): Promise<Response>;
20
21 // Implementation
22 function fetch(
23     url: string,
24     optionsOrMethod?: FetchOptions | "POST" | "PUT",
25     body?: any
26 ): Promise<Response> {
27     let method: string = "GET";
28     let headers: Record<string, string> = {
29         "Content-Type": "application/json"
30     };
31     let requestBody: any = undefined;
32
33     // Parse arguments based on overload pattern
34     if (typeof optionsOrMethod === "object") {
35         // Overload 2: options object
36         method = optionsOrMethod.method || "GET";
37         headers = { ...headers, ...optionsOrMethod.headers };
38         requestBody = optionsOrMethod.body;
39     } else if (typeof optionsOrMethod === "string") {
40         // Overload 3: method and body
41         method = optionsOrMethod;
42         requestBody = body;
43     }
44     // else: Overload 1 (defaults are already set)
45
46     console.log(`${method} ${url}`);
47     console.log("Headers:", headers);
48     if (requestBody) {
49         console.log("Body:", requestBody);
50     }
```

```
54     status: 200,  
55     data: { success: true }  
56   });  
57 }  
58  
59 // Test all overload patterns  
60 fetch("https://api.example.com/users");  
61  
62 fetch("https://api.example.com/users", {  
63   method: "GET",  
64   headers: { "Authorization": "Bearer token" }  
65 });  
66  
67 fetch("https://api.example.com/users", "POST", {  
68   name: "Alice",  
69   email: "alice@example.com"  
70 });
```

</details>



Summary

In this lesson, you learned:

- ✓ How to use rest parameters (`...args`) to accept variable-length arguments
- ✓ How to type rest parameters with arrays and tuples
- ✓ How to create function overloads for different call signatures
- ✓ When to use overloads vs union types
- ✓ How to combine rest parameters with function overloads
- ✓ Best practices for writing maintainable overloaded functions

Rest parameters and function overloads give you powerful tools for creating flexible, type-safe APIs that adapt to different usage patterns while maintaining full TypeScript type checking.



Next Steps

Mark as complete